計算機組織

Final Project

組員:

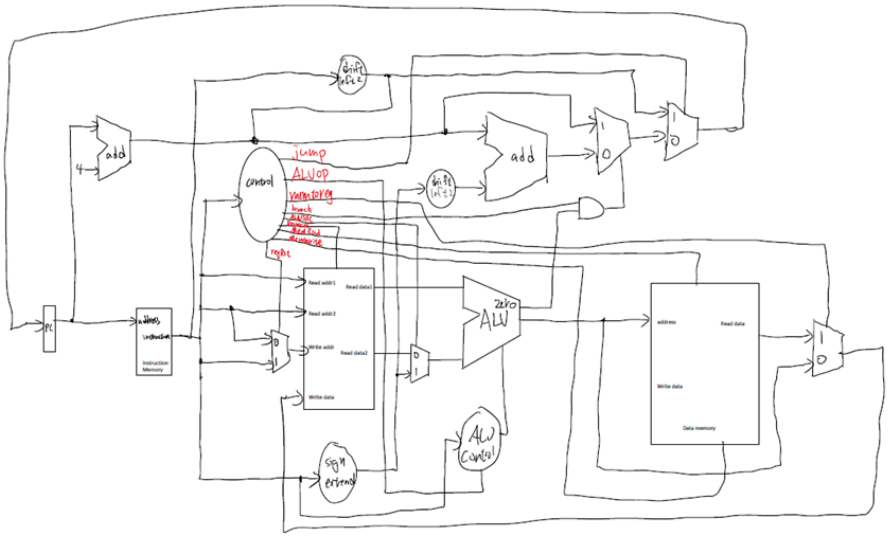
通訊3 Veli

電機3B Romi

電機3B Suri

A. Single cycle

1. 架構圖



Test code
addi $s1, $zero, 17 addi $s2, $zero, 19 sw $s1, 4($s2) sub $s3, $s1, $s2 addi $s2, $zero, -2 beq $s2, $s3, LABEL1 LABEL2: addi $s2, $zero, 19 lw $s1, 4($s2) j EXIT LABEL1: and $s5, $s2, $s3 or $t1, $s5, $s1 slt $s5, $s2, $s3

add $t2, $s5, $t1

j LABEL2

EXIT:

Instruction

```
memory[0] <= 32'b001000_00000_10001_0000000000010001; //addi $s1, $zero, 17
memory[1] <= 32'b001000_00000_10010_0000000000010011; //addi $s2, $zero, 19
memory[2] <= 32'b101011_10010_10001_0000000000000100; //sw $s1, 4($s2)
memory[3] <= 32'b000000_10001_10010_10011_00000_100010; //sub $s3, $s1, $s2
memory[4] <= 32'b001000_00000_10010_1111111111111110; //addi $s2, $zero, -2
memory[5] <= 32'b000100_10010_10011_0000000000000011; //beq $s2, $s3, LABEL1
memory[6] <= 32'b001000_00000_10010_0000000000010011; //addi $s2, $zero, 19
memory[7] <= 32'b100011_10010_10001_0000000000000100; //lw $s1, 4($s2)
memory[8] <= 32'b000010_00000000000000000000001110; //j  EXIT
memory[9] <= 32'b000000_10010_10011_10101_00000_100100; //and $s5, $s2, $s3
memory[10] <= 32'b000000_10101_10001_01001_00000_100101; //or $t1, $s5, $s1
memory[11] <= 32'b000000_10010_10011_10101_00000_101010; //slt $s5, $s2, $s3
memory[12] <= 32'b000000_10101_01001_01010_00000_100000; //add $t2, $s5, $t1
memory[13] <= 32'b000010_00000000000000000000000110; //j  LABEL2
```

2. 模擬結果

(表 1.)手動計算運用single cycle架構執行test code後，各個regester的值

register	$s1($17)	$s2($18)	$s3($19)	$s5($21)	$t1($9)	$t2($10)
data(dec)	17	19	-2	0	-1	-1

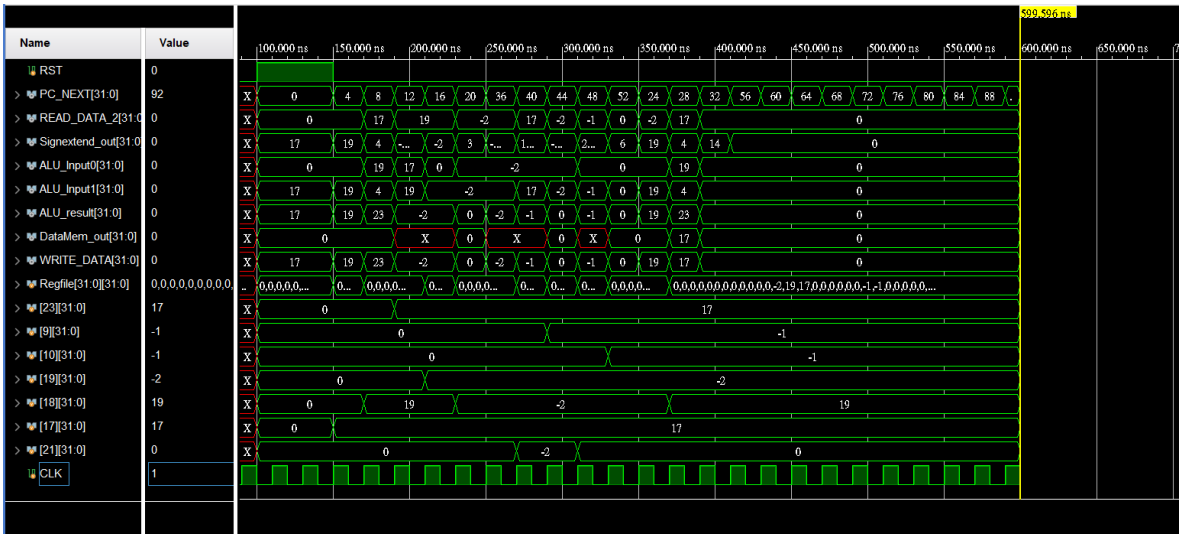
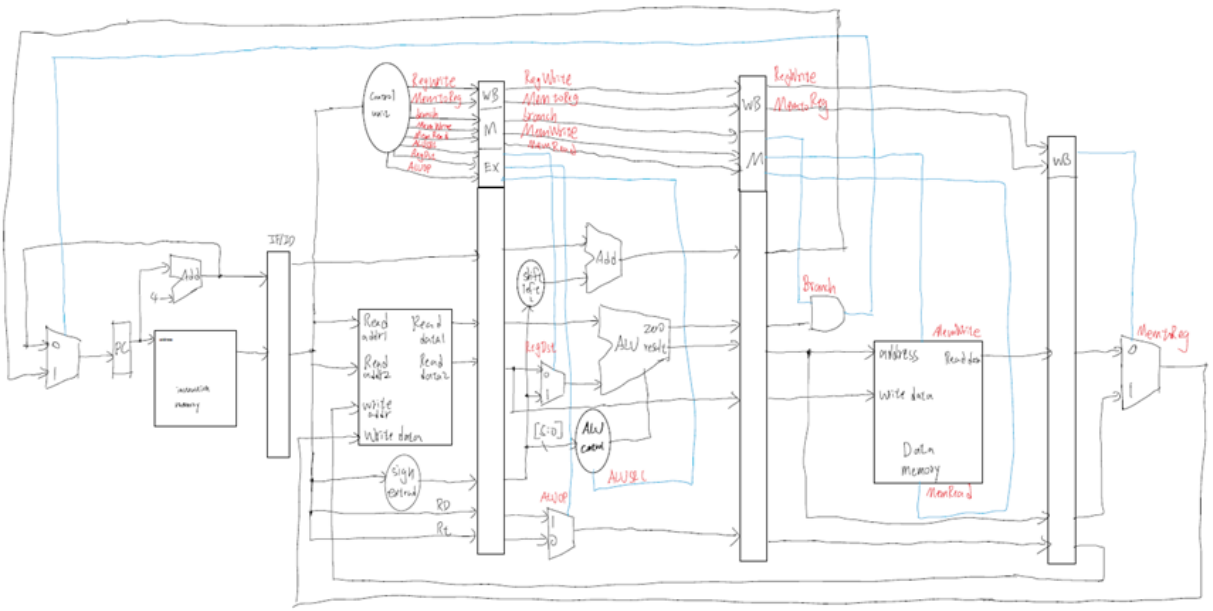


圖1. single cycle 模擬結果

B. Pipeline

1. 架構圖



Test code	
addi $s15 $zero 49 addi $s16 $zero 37 addi $s7 $zero 24 /addi $s8 $zero 25 sub $s17 $s15 $s16 add $s18 $s15 $s16	sw $16 4($15) and $9 $7 $8 beq $15 $16 Lable or $7 $8 $11/slt $20 $7 $8 lw $12 4($15)

instruction memeory

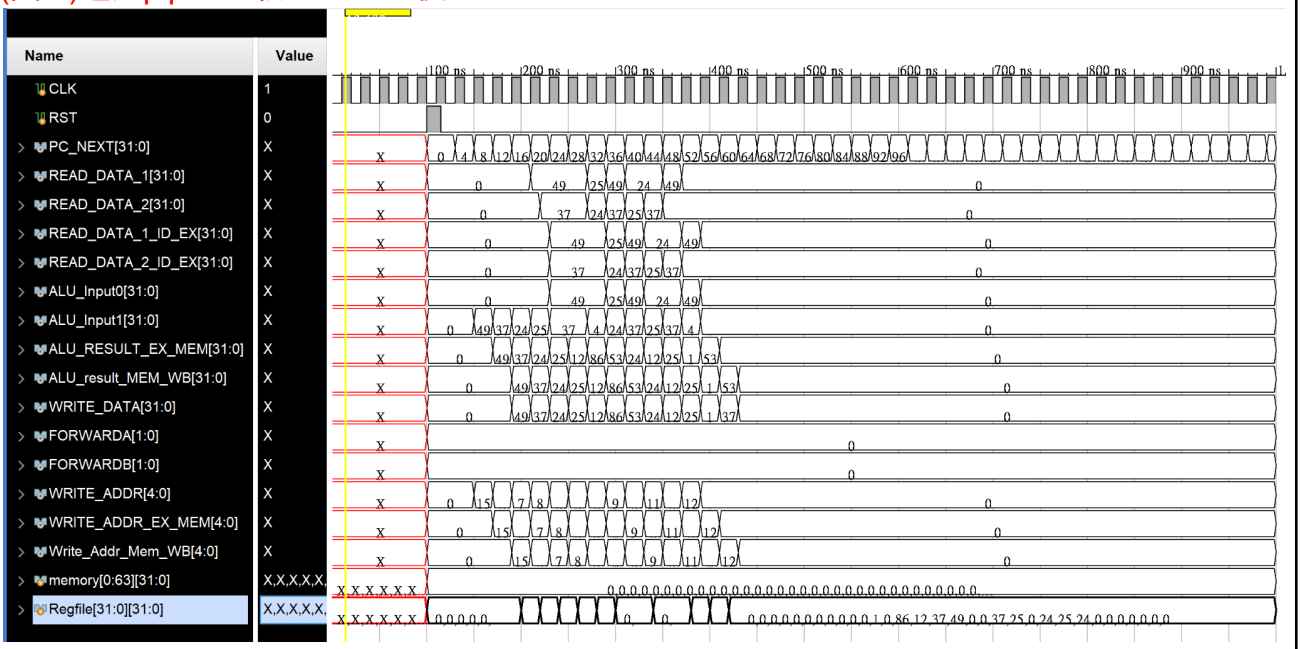
memory[0] <= 32'b001000_00000_01111_0000000000110001; //addi $15 $zero 49
memory[1] <= 32'b001000_00000_10000_0000000000100101; //addi $16 $zero 37
memory[2] <= 32'b001000_00000_00111_0000000000011000; //addi $7 $zero 24
memory[3] <= 32'b001000_00000_01000_0000000000011001; //addi $8 $zero 25
memory[4] <= 32'b000000_01111_10000_10001_00000_100010;//sub $17 $15 $16
memory[5] <= 32'b000000_01111_10000_10010_00000_100000;//add $18 $15 $16
memory[6] <= 32'b101011_01111_10000_0000000000000100;//sw $16 4($15)
memory[7] <= 32'b000000_01000_00111_01001_00000_100100;//and $9 $7 $8
memory[8] <= 32'b000100_01111_10000_0000000000000011; //beq $15 $16 Lable
memory[9] <= 32'b000000_00111_01000_01011_00000_100101;//or $7 $8 $11
memory[10] <= 32'b000000_00111_10000_10100_00000_101010;//slt $20 $7 $8
memory[11] <= 32'b100011_01111_01100_0000000000000100;//lw $12 4($15)

2. 模擬結果

(表 1.)手動計算運用hazard detect 的pipeline執行test code後, 各個regester的值

regester	$7	$8	$9	$12	$15	16	$17	$18	20
data(dec)	24	25	24	37	49	37	12	86	1

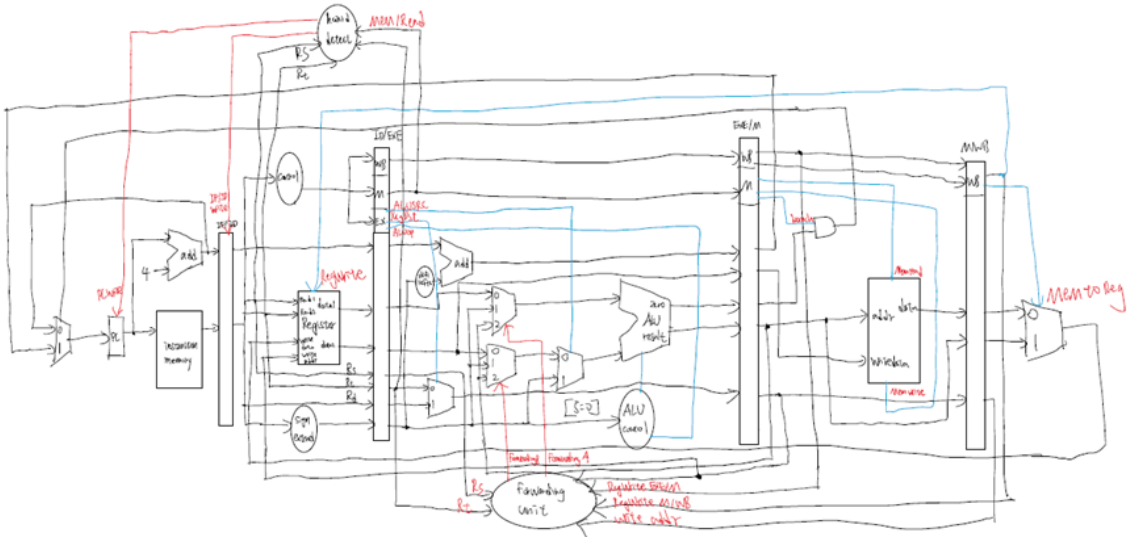
(圖2.)運用pipeline執test code後的waveform



說明:如圖1.的Regfile所示，運算結果和理想一模一樣

C. Datapath with hazard detection

1. 架構圖



Test code

addi $2, 0, 2 addi $4, 0, 4 addi $5, 0, 5 addi $6, 0, 6 sub $3, $2, $4 and $13, $3, $6 or $14, $5, $3 add $15, $3, $3 sw $15, 16($3)	addi $2, 0, 2 addi $4, 0, 4 addi $5, 0, 5 addi $6, 0, 6 sub $3, $2, $4 and $13, $3, $6 or $14, $5, $3 add $15, $3, $3 sw $15, 16($3)

Instruction Memory

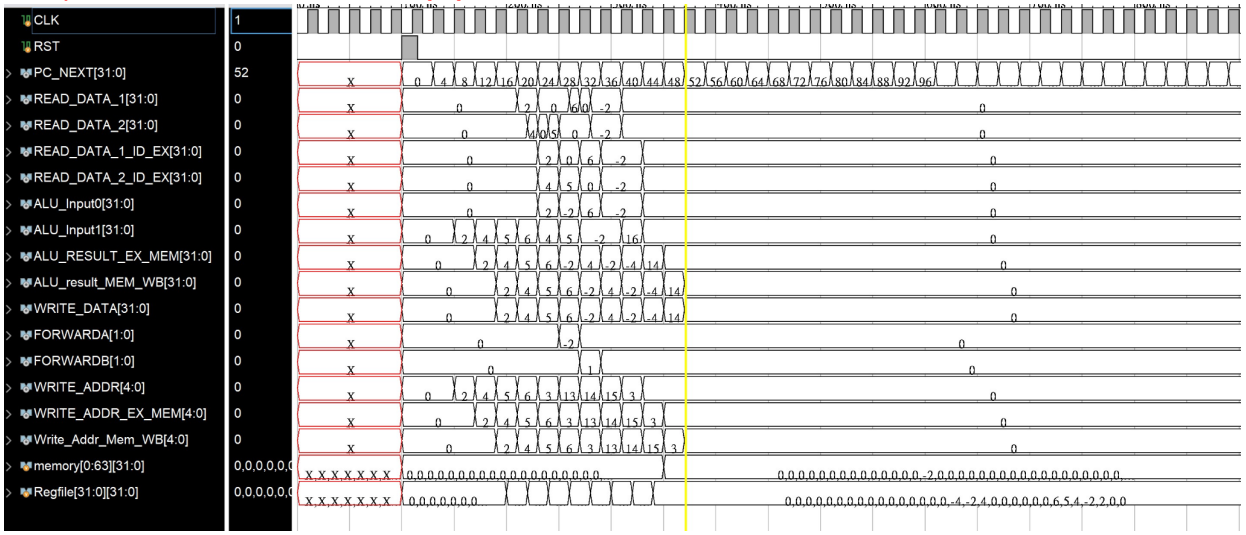
```
memory[0] <= 32'b001000_00000_00010_0000000000000010;// addi  $2  0 2
memory[1] <= 32'b001000_00000_00100_0000000000000100;//addi $4  0 4
memory[2] <= 32'b001000_00000_00101_0000000000000101;//addi $5 0 5
memory[3] <= 32'b001000_00000_00110_0000000000000110;//addi $6 0 6
memory[4] <= 32'b000000_00010_00100_00011_00000_100010;//sub $3 $2 $4
memory[5] <= 32'b000000_00011_00101_01101_00000_100100;//and $13 $3 $6
memory[6] <= 32'b000000_00110_00011_01110_00000_100101;//or $14 $5 $3
memory[7] <= 32'b000000_00011_00011_01111_00000_100000;//add $15 $3 $3
memory[8] <= 32'b101011_00011_00011_0000000000010000;// sw $15 16($3)
```

2. 模擬結果

(表 1.)手動計算運用hazard detect 的pipeline執行test code後，各個regester的值

regester	$2	$3	$4	$5	$6	$13	$14	$15
data(dec)	2	-2	4	5	6	4	-2	-4

(圖2.)運用hazard detect 的pipeline執行code 1.後的waveform



說明:如圖1.的Regfile所示，運算結果和理想一模一樣

pipeline 結構會造成前面的指令尚未寫入 register 後面的指令即要讀值的狀況，故需要加入 新的control unit 來預防 hazard 發生。每次運算時，會比較該指令要使用的 register 是否與前 1 個或前 2 個指令，寫入的 register 重複;若一致將抓前 1 個或前 2 個指令要寫回的結果拿來運算，避免抓成舊值做運算(若同時重複，需取前 1 個指令為運算)，而判斷條件的步驟如下:

步驟一、判斷是否 forwarding?

1. REGWRITE_EX_MEM 是否為 1 (是否可被寫入)
2. RD_MEM_WB 或 RD_EX_MEM 是否不等於 0 (被寫入的 register address 非 $zero)
3. RD_MEM_WB 或 RD_EX_MEM 是否等於 RS 或 RT

(是否未寫入即用到，發生 hazard，對應不同狀態得不同的 forwardingA、B 控制訊號)

步驟二、判斷是否啟動 hazard detection?
1. ID/EX.MemRead 是否為 1 (Memory 是否可讀新值)
2. ID/EX.RegisterRt 是否等於 IF/ID.RegisterRs 或 IF/ID.RegisterRt
(若為是, 控制訊號均歸零;否則繼續運算)

D. 小組分工表

	Romi	Suri	Veli
比例分配	33.33%	33.33%	33.33%

我們一起把一個階段完成, 再進入下一個階段:

single cycle 完成, 再做 pipeline , 最後一起做hazard detection